

Python Interview Questions & Answers

A complete, example-driven reference for Data Analyst & Data Engineering interviews

Prepared for Gaurav · Interview Preparation Guide

Table of Contents

- 1. 1. Python Basics
- 2. 2. Data Structures & Strings
- 3. 3. Functions, Lambda & Decorators
- 4. 4. OOP in Python
- 5. 5. Iterators, Generators & Exception Handling
- 6. 6. Pandas & Data Handling
- 7. 7. Miscellaneous / Tricky Interview Questions

Note: Examples in the Pandas & Data Handling section assume a typical sales/orders dataset with columns such as OrderID, CustomerID, CustomerName, Region, Amount, Price, and Quantity — consistent with the style used in the companion SQL Interview Q&A guide.

1. Python Basics

Q1. What is Python and what are its key features?

Python is a high-level, interpreted, dynamically-typed, general-purpose programming language. Key features: easy-to-read syntax, dynamic typing, automatic memory management (garbage collection), extensive standard library, support for multiple paradigms (procedural, object-oriented, functional), and a huge ecosystem of third-party packages.

```
print("Hello, World!")

# Dynamic typing - no need to declare variable types
x = 10          # int
x = "Gaurav"    # now a string, same variable
```

Q2. What is the difference between a list, tuple, and set?

List – ordered, mutable, allows duplicates.

Tuple – ordered, immutable, allows duplicates, generally faster than a list.

Set – unordered, mutable, does not allow duplicates, optimized for membership checks.

Dict – key-value pairs, ordered (Python 3.7+), keys must be unique.

```
my_list = [1, 2, 2, 3]      # mutable, ordered, duplicates OK
my_tuple = (1, 2, 2, 3)     # immutable, ordered
my_set = {1, 2, 2, 3}       # -> {1, 2, 3}, no duplicates
my_dict = {"a": 1, "b": 2}  # key-value pairs
```

Q3. What is the difference between deep copy and shallow copy?

A shallow copy creates a new object but inserts references to the same nested objects found in the original, so changes to nested/mutable elements affect both. A deep copy recursively copies everything, so the new object is fully independent.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
shallow[0][0] = 99
print(original)  # [[99, 2], [3, 4]] -- original also changed!

deep = copy.deepcopy(original)
deep[0][0] = 100
print(original)  # unaffected by change to 'deep'
```

Q4. What is the difference between '==' and 'is'?

'==' checks value equality (do the objects hold the same data), while 'is' checks identity (do both names point to the exact same object in memory).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)  # True  -> same values
print(a is b)  # False -> different objects in memory
print(a is c)  # True  -> c refers to the same object as a
```

Q5. What are mutable and immutable data types in Python?

Mutable objects can be changed after creation (list, dict, set, bytearray). Immutable objects cannot be changed after creation; any "modification" creates a new object (int, float, str, tuple, frozenset, bool).

```
s = "hello"
s = s + " world"  # creates a NEW string object, doesn't modify original

lst = [1, 2, 3]
lst.append(4)      # modifies the SAME list object in place
print(lst)         # [1, 2, 3, 4]
```

Q6. What is PEP 8?

PEP 8 is Python's official style guide, covering naming conventions, indentation (4 spaces), line length, whitespace usage, and other conventions to keep code consistent and readable across projects.

```
# PEP 8 examples
def calculate_total(price, quantity):  # snake_case for functions/variables
    return price * quantity

class OrderProcessor:                  # PascalCase for classes
    MAX_ITEMS = 100                     # UPPER_CASE for constants
```

2. Data Structures & Strings

Q7. How do you reverse a string in Python?

The most Pythonic way is slicing with a step of -1. You can also use reversed() combined with join(), or a loop.

```
text = "python"
reversed_text = text[::-1]
print(reversed_text)    # nohtyp

# Alternative
reversed_text2 = "".join(reversed(text))
```

Q8. What is list comprehension? Give an example.

List comprehension is a concise way to create lists using a single line of code, often replacing a for-loop with append(). It can also include conditions.

```
squares = [x**2 for x in range(10)]
even_squares = [x**2 for x in range(10) if x % 2 == 0]

# Dict and set comprehensions work similarly
squares_dict = {x: x**2 for x in range(5)}
unique_lengths = {len(word) for word in ["hi", "python", "sql"]}
```

Q9. What is the difference between append() and extend() for lists?

append() adds its argument as a single element to the end of the list. extend() iterates over its argument and adds each element individually.

```
lst = [1, 2, 3]
lst.append([4, 5])
print(lst)  # [1, 2, 3, [4, 5]]  -- added as one nested element

lst2 = [1, 2, 3]
lst2.extend([4, 5])
print(lst2)  # [1, 2, 3, 4, 5]    -- added individually
```

Q10. How do you remove duplicates from a list while preserving order?

Using a plain `set()` removes duplicates but loses order. To preserve order, use `dict.fromkeys()` (Python 3.7+ dicts maintain insertion order) or a loop with a seen-set.

```
items = [3, 1, 2, 3, 1, 4]

unique_ordered = list(dict.fromkeys(items))
print(unique_ordered)    # [3, 1, 2, 4]
```

Q11. What is the difference between a shallow slice and using `.copy()`?

Both `list[:]` and `list.copy()` create a shallow copy of a list - they behave the same way for one-dimensional lists. For nested/multi-dimensional lists, both still only copy the outer list, so inner lists remain shared references (use `copy.deepcopy` for full independence).

```
original = [1, 2, 3]
copy1 = original[:]
copy2 = original.copy()
copy1.append(4)
print(original)    # [1, 2, 3] -- unaffected
```

Q12. What are Python string formatting methods?

Common approaches: % formatting (old style), `str.format()`, and f-strings (Python 3.6+, most readable and fastest).

```
name, score = "Gaurav", 95

print("Name: %s, Score: %d" % (name, score))    # old style
print("Name: {}, Score: {}".format(name, score)) # .format()
print(f"Name: {name}, Score: {score}")          # f-string (preferred)
```

3. Functions, Lambda & Decorators

Q13. What is the difference between `*args` and `**kwargs`?

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. They allow functions to accept a variable number of arguments.

```
def summarize(*args, **kwargs):
    print("Positional:", args)
    print("Keyword:", kwargs)

summarize(1, 2, 3, name="Gaurav", role="Analyst")
# Positional: (1, 2, 3)
# Keyword: {'name': 'Gaurav', 'role': 'Analyst'}
```

Q14. What is a lambda function? When would you use one?

A lambda is a small, anonymous, single-expression function defined with the `lambda` keyword. It's typically used for short throwaway functions passed to functions like `map()`, `filter()`, `sorted()`, or as a key function.

```
square = lambda x: x ** 2
print(square(5))    # 25

# Common use: sorting by a custom key
people = [("Amit", 25), ("Gaurav", 30), ("Ravi", 22)]
sorted_people = sorted(people, key=lambda person: person[1])
```

Q15. What are map(), filter(), and reduce()?

map() applies a function to every item in an iterable and returns an iterator of results. filter() keeps only items for which a function returns True. reduce() (from functools) cumulatively applies a function to reduce an iterable to a single value.

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, nums))      # [1, 4, 9, 16, 25]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
total = reduce(lambda a, b: a + b, nums)      # 15
```

Q16. What is a decorator in Python? Give an example.

A decorator is a function that wraps another function to extend or modify its behavior without changing its source code, using the @decorator syntax. Commonly used for logging, timing, authentication, and caching.

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)

slow_function()    # prints execution time automatically
```

Q17. What is the difference between a function argument's default value being mutable vs immutable?

A mutable default argument (like a list) is created only once when the function is defined, and is shared across all calls that don't override it — a classic Python gotcha. The safe pattern is to default to None and create the mutable object inside the function body.

```
# Buggy: default list is shared across calls
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1))    # [1]
print(add_item(2))    # [1, 2] -- unexpected! same list reused

# Fixed
def add_item_safe(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Q18. What is a closure in Python?

A closure is a function that remembers and can access variables from its enclosing (outer) scope even after the outer function has finished executing.

```
def make_multiplier(factor):
    def multiplier(x):
        return x * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5))    # 10
print(triple(5))    # 15
```

4. OOP in Python

Q19. What are the four pillars of OOP, and how does Python implement them?

Encapsulation – bundling data and methods, using naming conventions (`_protected`, `__private`) to restrict access.

Abstraction – hiding implementation details via abstract base classes.

Inheritance – a class can derive from another class to reuse behavior.

Polymorphism – the same method name can behave differently across classes.

```
class Animal:
    def speak(self):
        raise NotImplementedError

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

for animal in [Dog(), Cat()]:
    print(animal.speak()) # Polymorphism: same method, different behavior
```

Q20. What is the difference between `__init__` and `__new__`?

`__new__` is a static method that creates and returns a new instance (allocates memory) — it runs first. `__init__` initializes the already-created instance's attributes and returns None. `__new__` is rarely overridden except for special cases like singletons or immutable types.

```
class Point:
    def __new__(cls, *args, **kwargs):
        print("Creating instance")
        return super().__new__(cls)

    def __init__(self, x, y):
        print("Initializing instance")
        self.x = x
        self.y = y

p = Point(1, 2)
# Output:
# Creating instance
# Initializing instance
```

Q21. What is the difference between class methods, static methods, and instance methods?

An **instance method** takes 'self' and can access/modify instance state. A **class method** (`@classmethod`) takes 'cls' and works with the class itself, often used for alternative constructors. A **static method** (`@staticmethod`) takes neither — it's just a regular function logically grouped inside the class.

```
class Employee:
    company = "TechCorp"

    def __init__(self, name):
        self.name = name

    def show(self):
        # instance method
        return f"{self.name} works at {self.company}"

    @classmethod
    def change_company(cls, new_name):
        # class method
        cls.company = new_name

    @staticmethod
    def is_valid_name(name):
        # static method
        return len(name) > 0
```

Q22. What is method resolution order (MRO) and multiple inheritance?

When a class inherits from multiple parent classes, Python uses the C3 linearization algorithm to determine the order in which base classes are searched for a method — this order is called the MRO, viewable with `ClassName.__mro__` or `mro()`.

```
class A:
    def greet(self):
        return "Hello from A"

class B(A):
    def greet(self):
        return "Hello from B"

class C(A):
    def greet(self):
        return "Hello from C"

class D(B, C):
    pass

print(D().greet())          # "Hello from B" (follows MRO)
print(D.__mro__)            # (D, B, C, A, object)
```

Q23. What are dunder (magic) methods? Give examples.

Dunder methods (double underscore, e.g. `__init__`, `__str__`) let custom classes integrate with Python's built-in syntax and functions like `print()`, `len()`, `+`, and `==`.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __str__(self):
        return f"₹{self.amount}"

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __eq__(self, other):
        return self.amount == other.amount

a = Money(100)
b = Money(50)
print(a + b)          # ₹150 (uses __add__ and __str__)
print(a == Money(100)) # True (uses __eq__)
```

5. Iterators, Generators & Exception Handling

Q24. What is the difference between an iterator and a generator?

An iterator is any object implementing `__iter__()` and `__next__()`. A generator is a simpler way to create an iterator, defined with a function using 'yield' (or a generator expression), and it automatically implements the iterator protocol while keeping only one value in memory at a time (lazy evaluation).

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count_up_to(5):
    print(num)    # 1 2 3 4 5, generated one at a time

# Generator expression (memory-efficient, like list comprehension but lazy)
squares_gen = (x**2 for x in range(1000000))
```


Q25. Why use a generator instead of a list for large datasets?

Generators produce values lazily, one at a time, so they use constant memory regardless of dataset size — ideal for streaming large files or huge sequences, where building a full list upfront would consume too much memory.

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            # yields one line at a time, not loading whole file
            yield line.strip()

# Processes the file without ever holding it all in memory
for line in read_large_file("huge_log.txt"):
    process(line)
```

Q26. How does exception handling work in Python?

Python uses try/except blocks. 'try' contains code that might raise an exception, 'except' catches and handles specific exception types, 'else' runs if no exception occurred, and 'finally' always runs (cleanup code) regardless of outcome.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
except (TypeError, ValueError) as e:
    print(f"Type/Value error: {e}")
else:
    print("No error occurred")
finally:
    print("This always runs (e.g. closing a file/connection)")
```

Q27. How do you create and raise a custom exception?

Define a new class inheriting from Exception (or a more specific built-in exception), then use 'raise' to trigger it when a specific error condition occurs.

```
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}, balance is only {balance}")

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientBalanceError as e:
    print(e)
```

Q28. What is the 'with' statement used for?

The 'with' statement (context manager) ensures that setup and cleanup code runs automatically, even if an exception occurs — most commonly used for file handling to guarantee the file is closed.

```
with open("data.txt", "r") as f:
    content = f.read()
# file is automatically closed here, even if an error occurred above

# Custom context manager
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Elapsed: {__import__('time').time() - self.start:.2f}s")

with Timer():
    sum(range(1000000))
```

6. Pandas & Data Handling

Q29. How do you read and inspect a CSV file with pandas?

`pd.read_csv()` loads data into a `DataFrame`. Common inspection methods: `head()`, `info()`, `describe()`, `shape`, and `dtypes` to quickly understand the data before analysis.

```
import pandas as pd

df = pd.read_csv("sales_data.csv")
print(df.head())          # first 5 rows
print(df.info())          # column types, non-null counts
print(df.describe())      # summary statistics for numeric columns
print(df.shape)           # (rows, columns)
```

Q30. How do you handle missing values in a pandas DataFrame?

Use `isnull()/isna()` to detect missing values, `dropna()` to remove rows/columns with missing data, and `fillna()` to impute them with a value, mean, median, or forward/backward fill.

```
df.isnull().sum()          # count missing values per column
df.dropna()               # drop rows with any missing value
df.dropna(subset=["Amount"]) # drop only if 'Amount' is missing
df["Amount"].fillna(df["Amount"].mean(), inplace=True) # fill with mean
df.fillna(method="ffill")  # forward-fill
```

Q31. What is the difference between `loc[]` and `iloc[]`?

`loc[]` selects rows/columns by label (name), and the end of a slice is inclusive. `iloc[]` selects by integer position, like standard Python indexing, and the end of a slice is exclusive.

```
df.loc[0:2, "CustomerName"] # rows labeled 0-2 (inclusive), by column name
df.iloc[0:2, 1]              # rows at position 0-1 (exclusive of 2), 2nd column
df.loc[df["Amount"] > 1000]  # boolean/label-based filtering
```

Q32. How do you group and aggregate data in pandas?

`groupby()` splits the `DataFrame` into groups based on column values, and aggregate functions (sum, mean, count, agg with multiple functions) are then applied per group, similar to SQL's GROUP BY.

```
summary = df.groupby("Region")["Sales"].sum().reset_index()

# Multiple aggregations at once
summary2 = df.groupby("Region").agg(
    TotalSales=("Sales", "sum"),
    AvgSales=("Sales", "mean"),
    OrderCount=("OrderID", "count")
).reset_index()
```

Q33. How do you merge/join two DataFrames?

`pd.merge()` combines `DataFrames` similarly to SQL joins, using the 'how' parameter (inner, left, right, outer) and 'on' to specify the join key(s).

```
orders = pd.DataFrame({"OrderID": [1, 2], "CustomerID": [101, 102], "Amount": [500, 700]})
customers = pd.DataFrame({"CustomerID": [101, 102], "Name": ["Amit", "Gaurav"]})

merged = pd.merge(orders, customers, on="CustomerID", how="left")
print(merged)
```

Q34. What is the difference between `apply()`, `map()`, and `applymap()`?

`map()` works element-wise on a Series only. `apply()` works on a Series or DataFrame (row-wise or column-wise, via `axis=`). `applymap()` applies a function to every element of an entire DataFrame.

```
df["Category"] = df["Amount"].map(lambda x: "High" if x > 1000 else "Low")

df["Total"] = df.apply(lambda row: row["Price"] * row["Quantity"], axis=1)

df_numeric = df[["Price", "Quantity"]].applymap(lambda x: round(x, 2))
```

7. Miscellaneous / Tricky Interview Questions

Q35. What is the Global Interpreter Lock (GIL)?

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core systems. It simplifies memory management but limits true parallelism for CPU-bound multi-threaded code; multiprocessing (separate processes) is used to achieve real parallelism instead.

```
from multiprocessing import Pool

def square(x):
    return x * x

if __name__ == "__main__":
    with Pool(4) as p:
        results = p.map(square, range(10))    # runs across 4 separate processes
```

Q36. What is the difference between multithreading and multiprocessing in Python?

Multithreading runs multiple threads within the same process, sharing memory, but is limited by the GIL for CPU-bound tasks (still useful for I/O-bound tasks like network calls). Multiprocessing runs separate processes with separate memory spaces, bypassing the GIL, making it better for CPU-bound tasks.

```
import threading, multiprocessing

def task():
    print("running")

# Good for I/O-bound work (e.g. API calls, file I/O)
t = threading.Thread(target=task)

# Good for CPU-bound work (e.g. heavy computation)
p = multiprocessing.Process(target=task)
```

Q37. What is the difference between a module and a package?

A module is a single `.py` file containing Python code (functions, classes, variables). A package is a directory containing multiple modules along with an `__init__.py` file (optional in Python 3.3+) that marks it as a package, allowing hierarchical organization.

```
# my_package/
#  __init__.py
#  utils.py
#  models.py

from my_package import utils
from my_package.models import Employee
```

Q38. What are Python's built-in data types for numbers?

int (arbitrary precision integers), float (double-precision floating point), complex (complex numbers with real and imaginary parts), and Decimal/Fraction from the standard library for precise or exact arithmetic.

```
x = 10          # int
y = 3.14        # float
z = 2 + 3j      # complex

from decimal import Decimal
price = Decimal("19.99") # avoids floating-point rounding errors
```

Q39. What is duck typing in Python?

Duck typing means Python cares about whether an object has the right methods/behavior, not its actual class — "if it walks like a duck and quacks like a duck, it's a duck." This enables flexible, polymorphic code without requiring explicit inheritance.

```
class Duck:
    def sound(self):
        return "Quack"

class Dog:
    def sound(self):
        return "Woof"

def make_sound(animal):
    print(animal.sound()) # works for ANY object with a .sound() method

make_sound(Duck())
make_sound(Dog())
```

Q40. What is the difference between '@staticmethod' and a regular module-level function?

Functionally similar, but a static method is namespaced inside a class for organizational purposes (logically related to the class) and accessed via ClassName.method(), while a module-level function is simply a standalone function.

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

print(MathUtils.add(2, 3)) # accessed through the class namespace
```

Q41. How does Python manage memory?

Python uses automatic memory management via reference counting (each object tracks how many references point to it) combined with a cyclic garbage collector to detect and clean up reference cycles that reference counting alone can't catch.

```
import sys
import gc

x = [1, 2, 3]
print(sys.getrefcount(x)) # current reference count

gc.collect() # manually trigger garbage collection
```

Q42. What is the difference between range() and xrange() (Python 2) / how does range() work in Python 3?

In Python 3, range() itself returns a lazy, memory-efficient sequence object (similar to the old Python 2 xrange()) rather than a fully materialized list, so it's efficient even for very large ranges without needing a separate xrange.

```
r = range(1000000)
print(type(r)) # <class 'range'> -- not a list, memory-efficient
print(list(r[:5])) # [0, 1, 2, 3, 4] -- only materialized when needed
```